Міністерство освіти і науки України

Національний технічний університет України

«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Лабораторна робота №1

з дисципліни «Бази даних»

Виконав:
Степаненко Денис
студент групи ІМ-051
номер у списку групи: 16

Перевірив:

Хмельницький Арсеній Андрійович

Київ 2026

Короткий виклад вимог

Завдання:

Є відеохостинг-платформа VideoHub (аналог YouTube). По ній треба зробити базу даних. Ця БД міститиме інформацію про користувачів, відео, коментарі, лайки та підписки. Потрібно спроектувати концептуальну модель бази даних — ER-діаграму, яка показує всі сутності, їхні атрибути та зв'язки між ними. На основі цієї діаграми в наступних лабораторних буде створена фізична схема в PostgreSQL.

Платформа повинна мати можливість додавати нові дані, редагувати наявні та видаляти контент. Кожен користувач може бути як автором відео (власником каналу), так і глядачем — дивитись відео, залишати коментарі, ставити лайки та підписуватись на інші канали. Також платформа повинна підтримувати авторизацію через Google OAuth та класичну авторизацію за паролем.

Потреби зацікавлених сторін:

Платформа повинна зберігати інформацію про користувачів (їхні профілі з аватаром, банером та описом каналу), відео (з метаданими: назва, опис, URL файлу, мініатюра, лічильник переглядів), коментарі до відео (текст та дата), лайки користувачів на відео, та підписки користувачів на інших користувачів (канали). Система повинна підтримувати додавання, редагування та видалення контенту. При видаленні користувача повинні каскадно видалятись всі пов'язані дані — його відео, коментарі, лайки та підписки.

Бізнес-правила та дані для зберігання:

- Кожен користувач має унікальний email та username — це гарантує, що два користувачі не можуть мати однаковий email або нікнейм.
- Користувач може авторизуватись через Google (google_id) або за паролем — підтримка OAuth та класичної авторизації.
- Кожен користувач має аватар, банер та опис каналу (необов'язково) — ці поля можуть бути порожніми при реєстрації.
- Кожне відео має назву, опис (необов'язково), URL файлу, мініатюру (thumbnail) — URL вказує на файл у сховищі S3/CDN.
- Відео має лічильник переглядів (за замовчуванням 0) — збільшується при кожному перегляді.
- Відео належить одному користувачу (автору) — зв'язок один-до-багатьох.
- Користувач може залишати коментарі до відео — один коментар належить одному юзеру та одному відео.
- Коментар містить текст та дату створення — дата встановлюється автоматично.
- Користувач може поставити лайк на відео (один лайк на відео від одного користувача) — обмеження UNIQUE.

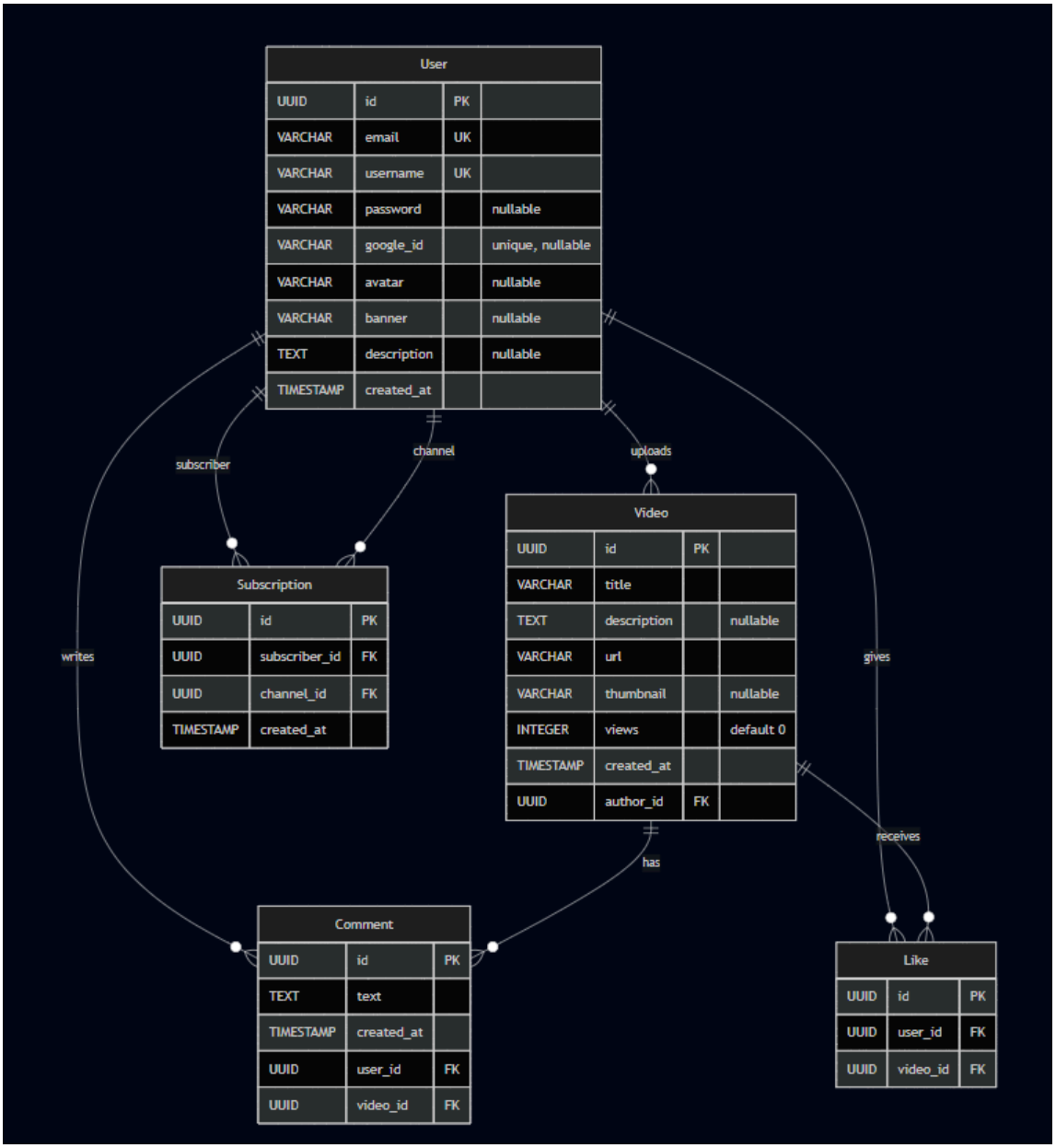
- Користувач може підписатись на іншого користувача (канал) — самореференсний зв'язок M:N.
- Підписка унікальна: один користувач може підписатись на іншого лише один раз — обмеження UNIQUE.
- Кожен запис має дату створення (created_at) — встановлюється автоматично при вставці.
- Користувач не може підписатись сам на себе — обмеження CHECK.

Нефункціональні вимоги:

Окрім функціональних вимог (що система має робити), до платформи висуваються і нефункціональні — якості, якими система повинна володіти:

- **Продуктивність.** Типові операції платформи — це короткі транзакції (OLTP): відкрити відео, залишити коментар, поставити лайк, підписатись на канал. Вони мають виконуватись швидко — за десятки мілісекунд. Лічильник переглядів збільшується атомарним UPDATE, який не блокує читання стрічки.
- **Масштабованість.** Кількість користувачів і відео може сильно зростати. Лічильник views зберігається як готове значення (денормалізація), щоб не рахувати COUNT по таблиці переглядів при кожному показі. UUID-ідентифікатори дозволяють генерувати id розподілено, без центрального лічильника, — це запас на майбутню масштабованість.
- **Цілісність даних.** У базі не повинно бути "сирітських" записів — коментаря без відео, лайка без користувача. Це гарантують зовнішні ключі з ON DELETE CASCADE. Унікальність пари у лайках та підписках забезпечена композитними первинними ключами — дублікат неможливий на рівні структури, а не лише застосунку.
- **Узгодженість бізнес-правил.** Правила не залежать від клієнта: обмеження CHECK (обов'язковий спосіб входу — пароль або Google, невід'ємні перегляди, заборона підписки самого на себе) виконуються самою базою для будь-якого джерела запису.
- **Безпека.** Паролі зберігаються лише у вигляді хешу — відкритий пароль у базі не лежить. Ідентифікатори UUID не вгадуються у публічних API, на відміну від послідовних чисел. Авторизація через Google OAuth зменшує кількість паролів, які платформа взагалі зберігає.
- **Надійність.** Операції, що зачіпають кілька таблиць, виконуються атомарно: транзакція або фіксується повністю, або повністю відкочується — половинчастого стану не буває. Видалення користувача каскадно прибирає весь його контент однією операцією, без залишків.
- **Підтримуваність.** Схема бази версіонується міграціями (у лабораторній 6 — Prisma ORM). Скрипт schema.sql ідемпотентний: базу можна видалити й відтворити з нуля в будь-який момент, що спрощує розгортання та тестування.

Діаграма ER



Список сутностей з їх атрибутами та пояснення кожного зв'язку прозою

User {

- id: PK — унікальний ідентифікатор користувача (UUID, генерується автоматично)
- email: UNIQUE, NOT NULL — email адреса, унікальна по всій системі
- username: UNIQUE, NOT NULL — нікнейм користувача, унікальний
- password: NULLABLE — хеш пароля (NULL, якщо авторизація через Google)
- google_id: UNIQUE, NULLABLE — ідентифікатор Google OAuth (NULL, якщо авторизація за паролем)
- avatar: NULLABLE — URL зображення аватара
- banner: NULLABLE — URL зображення банера каналу
- description: NULLABLE — текстовий опис каналу

- created_at: NOT NULL, DEFAULT now() — дата реєстрації користувача

}

Video {

- id: PK — унікальний ідентифікатор відео (UUID)
- title: NOT NULL — назва відео, обов'язкове поле
- description: NULLABLE — опис відео, необов'язкове
- url: NOT NULL — посилання на відеофайл у сховищі S3/CDN
- thumbnail: NULLABLE — URL мініатюри відео
- views: DEFAULT 0 — лічильник переглядів, починається з нуля
- created_at: NOT NULL, DEFAULT now() — дата завантаження відео
- author_id: FK → User(id), NOT NULL — зовнішній ключ на автора відео

}

Comment {

- id: PK — унікальний ідентифікатор коментаря (UUID)
- text: NOT NULL — текст коментаря, обов'язкове поле
- created_at: NOT NULL, DEFAULT now() — дата створення коментаря
- user_id: FK → User(id), NOT NULL — автор коментаря (хто написав)
- video_id: FK → Video(id), NOT NULL — відео, до якого належить коментар

}

Like {

- id: PK — унікальний ідентифікатор лайка (UUID)
- user_id: FK → User(id), NOT NULL — хто поставив лайк
- video_id: FK → Video(id), NOT NULL — на яке відео поставлено лайк
- UNIQUE(user_id, video_id) — один користувач може поставити лише один лайк на одне відео

}

Subscription {

- id: PK — унікальний ідентифікатор підписки (UUID)
- subscriber_id: FK → User(id), NOT NULL — хто підписався
- channel_id: FK → User(id), NOT NULL — на кого підписався (канал)
- created_at: NOT NULL, DEFAULT now() — дата підписки
- UNIQUE(subscriber_id, channel_id) — не можна підписатись двічі на один канал
- CHECK(subscriber_id ≠ channel_id) — не можна підписатись сам на себе

}

Пояснення кожного зв'язку прозою:

- **User – Video:** один користувач (автор) може завантажити багато відео, але кожне відео належить лише одному автору. Зв'язок один-до-багатьох (1:N). Реалізовано через зовнішній ключ author_id у таблиці Video, що посилається на первинний ключ id у таблиці User. При видаленні користувача каскадно видаляються всі його відео (ON DELETE CASCADE).

- **User – Comment – Video:** один користувач може залишити багато коментарів, і одне відео може мати багато коментарів. Коментар належить одночасно одному користувачу (автору) та одному відео. Це один-до-багатьох з обох боків, оскільки Comment є самостійною сутністю з власним первинним

ключем та атрибутом text. Реалізовано через два зовнішні ключі: user_id → User(id) та video_id → Video(id). При видаленні відео або користувача коментарі видаляються каскадно.

- **User – Like – Video:** один користувач може поставити лайки на багато відео, і одне відео може отримати багато лайків від різних користувачів. Обмеження UNIQUE(user_id, video_id) гарантує, що один користувач може поставити лише один лайк на одне відео. Реалізовано через два зовнішні ключі: user_id → User(id) та video_id → Video(id). При видаленні відео або користувача лайки видаляються каскадно.
- **User – Subscription – User:** один користувач (subscriber) може підписатись на багато каналів, і один канал може мати багато підписників. Це багато-до-багатьох (M:N), реалізовано через проміжну таблицю Subscription. Обидва зовнішні ключі (subscriber_id та channel_id) посилаються на ту саму таблицю User — це самореференсний зв'язок. Обмеження UNIQUE(subscriber_id, channel_id) забороняє повторну підписку на один канал. Обмеження CHECK(subscriber_id ≠ channel_id) забороняє підписку самого на себе.

Будь які припущення чи обмеження

- Пароль може бути NULL, якщо користувач авторизується лише через Google. Це означає, що поле password не має обмеження NOT NULL, оскільки частина користувачів використовує OAuth-авторизацію і не має пароля в системі.
- google_id унікальний і може бути NULL для користувачів з паролем. Користувачі, що зареєструвались за паролем, мають google_id = NULL. Користувачі через Google мають password = NULL.
- Лічильник переглядів збільшується при кожному перегляді відео. Початкове значення — 0 (DEFAULT 0). Збільшення відбувається через UPDATE запит в додатку.
- Видалення відео каскадно видаляє його коментарі та лайки. Це забезпечується обмеженням ON DELETE CASCADE на зовнішніх ключах у таблицях Comment та Like.
- Видалення користувача каскадно видаляє його відео, коментарі, лайки та підписки. Каскадне видалення поширюється ланцюжком: User → Video → Comment, Like. Також видаляються підписки, де користувач був підписником або каналом.
- URL відео зберігає посилання на файл у сховищі (S3/CDN), сама БД не зберігає відеофайли. База даних зберігає лише метадані відео та лічильники.
- Дата створення (created_at) встановлюється автоматично при вставці запису через DEFAULT now(). Це гарантує, що кожен запис має коректну дату створення без явної вказівки в INSERT-запиті.
- Email та username унікальні по всій системі. Обмеження UNIQUE на ці поля гарантує, що два користувачі не можуть мати однаковий email або username.
- Користувач не може підписатись сам на себе. Обмеження CHECK(subscriber_id ≠ channel_id) на таблиці Subscription гарантує, що subscriber_id і channel_id не можуть бути однаковими.
- Всі первинні ключі — це UUID (унікальний 128-бітний ідентифікатор). UUID генерується автоматично при створенні запису. Переваги UUID над SERIAL: глобальна унікальність, безпека (не можна вгадати ID перебором), зручність для розподілених систем.